

SQL Queries:

1. Salary between 50,000 and 80,000

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 80000
ORDER BY salary DESC, emp_name ASC;
```

2. Multi-condition Employee Search

```
SELECT emp_id, emp_name, dept_id, salary, manager_id
FROM employees
WHERE (dept_id = 10 AND salary > 55000)
      OR (dept_id = 20 AND salary >= 70000)
AND manager_id IS NOT NULL;
```

Safer version with brackets:

```
SELECT emp_id, emp_name, dept_id, salary, manager_id
FROM employees
WHERE (
      (dept_id = 10 AND salary > 55000)
      OR
      (dept_id = 20 AND salary >= 70000)
)
AND manager_id IS NOT NULL;
```

3. Pattern Matching

Names containing a after the first character and not ending with n:

```
SELECT emp_id, emp_name
FROM employees
```

```
WHERE emp_name LIKE '_%a%'
```

```
AND emp_name NOT LIKE '%n'
```

```
ORDER BY emp_name;
```

4. Salary Classification using CASE

```
SELECT emp_name, salary,
```

```
    CASE
```

```
        WHEN salary >= 75000 THEN 'High'
```

```
        WHEN salary >= 55000 THEN 'Medium'
```

```
        ELSE 'Standard'
```

```
    END AS salary_band
```

```
FROM employees
```

```
ORDER BY salary_band, salary DESC;
```

5. Department Salary Statistics

```
SELECT dept_id,
```

```
    COUNT(*) AS employee_count,
```

```
    MIN(salary) AS min_salary,
```

```
    MAX(salary) AS max_salary,
```

```
    AVG(salary) AS avg_salary,
```

```
    SUM(salary) AS total_salary
```

```
FROM employees
```

```
GROUP BY dept_id
```

```
ORDER BY avg_salary DESC;
```

6. Strong Departments

```
SELECT dept_id,
```

```
    COUNT(*) AS employee_count,
```

```
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING COUNT(*) >= 2
AND AVG(salary) > 55000;
```

7. Project Budget Analysis

```
SELECT dept_id,
       COUNT(*) AS project_count,
       SUM(budget) AS total_budget
FROM projects
GROUP BY dept_id
HAVING SUM(budget) >= 300000;
```

JOIN Questions

8. Employee Department Report

```
SELECT e.emp_id,
       e.emp_name,
       d.dept_name,
       d.location,
       e.salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
WHERE e.salary >= 60000
ORDER BY d.dept_name, e.salary DESC;
```

9. All Departments With Employee Counts

```
SELECT d.dept_id,  
       d.dept_name,  
       COUNT(e.emp_id) AS employee_count  
FROM departments d  
LEFT JOIN employees e  
ON d.dept_id = e.dept_id  
GROUP BY d.dept_id, d.dept_name;
```

Important: Use COUNT(e.emp_id), not COUNT(*), because we want 0 for departments with no employees.

10. Employee and Manager Names

```
SELECT e.emp_name AS employee_name,  
       e.salary AS employee_salary,  
       m.emp_name AS manager_name,  
       m.salary AS manager_salary  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.emp_id  
ORDER BY m.emp_name, e.salary DESC;
```

11. Department Project Coverage

```
SELECT d.dept_id,  
       d.dept_name,  
       COUNT(p.project_id) AS project_count,  
       COALESCE(SUM(p.budget), 0) AS total_budget  
FROM departments d
```

```
LEFT JOIN projects p
ON d.dept_id = p.dept_id
GROUP BY d.dept_id, d.dept_name;
```

12. Employees in Project-Active Departments

```
SELECT e.emp_name,
       d.dept_name,
       p.project_name,
       e.salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
JOIN projects p
ON d.dept_id = p.dept_id;
```

Subqueries

13. Highest Paid Employee(s)

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

14. Employees Above Company Average

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > (
```

```
SELECT AVG(salary)
FROM employees
)
ORDER BY salary DESC;
```

15. Employees Above Their Department Average

```
SELECT emp_id, emp_name, dept_id, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE dept_id = e.dept_id
);
```

16. Highest Paid Employee Per Department

```
SELECT emp_id, emp_name, dept_id, salary
FROM employees e
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE dept_id = e.dept_id
);
```

This also handles ties.

17. Departments Without Projects

```
SELECT dept_id, dept_name, location
FROM departments d
WHERE NOT EXISTS (
```

```
SELECT 1
FROM projects p
WHERE p.dept_id = d.dept_id
);
```

Harder Subqueries

18. More Than Every HR Employee

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > ALL (
    SELECT e.salary
    FROM employees e
    WHERE e.dept_id = (
        SELECT dept_id
        FROM departments
        WHERE dept_name = 'HR'
    )
);
```

Here we don't hard-code 30.

19. Employees in Above-Average Departments

```
SELECT emp_id, emp_name, dept_id, salary
FROM employees
WHERE dept_id IN (
    SELECT dept_id
    FROM employees
```

```
GROUP BY dept_id
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
)
);
```

20. Closest to Company Average

This one is the hardest. A simple way is:

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE ABS(salary - (
    SELECT AVG(salary)
    FROM employees
)) = (
    SELECT MIN(ABS(salary - (
        SELECT AVG(salary)
        FROM employees
    )))
FROM employees
);
```